NASH MESH

MESHCORE

Field Cards

A field guide to running on the Nashville MeshCore mesh — radio, routing, messaging, access, and repair.

23 cards · settings → troubleshooting

MESHCORE · FIELD CARD 01

Radio Settings

`set radio <freq>,<bw>,<sf>,<cr>`

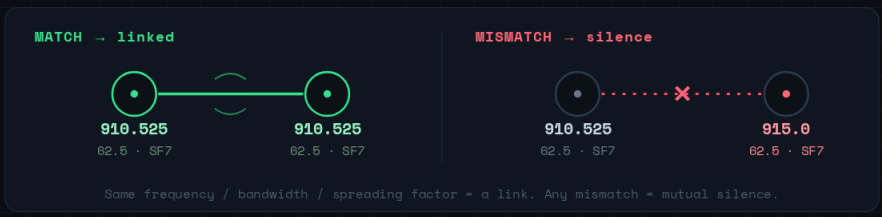
The five values that put you on NashMesh. Get one wrong and you're transmitting into the void — the mesh literally can't hear you, and you can't hear it.

PRESET **USA/Canada** · FREQ **910.525 MHz** · BW **62.5 kHz** · RULE **must match the mesh**

1 Match exactly, or you're invisible

LoRa radios only hear each other on an identical channel.

Frequency, bandwidth and spreading factor together define a LoRa **channel**. If even one differs from the rest of the mesh, your transmissions are unreadable — you'll hear nothing and no one hears you. These aren't preferences; they're the shared address everyone agrees on.



2 The NashMesh channel

Copy these on every node — companion or repeater.

Preset	USA / Canada
Frequency	910.525 MHz
Bandwidth	62.5 kHz · narrow = more reach, slower
Spreading Factor	7 → see Card 04
Coding Rate	5 → see Card 05

3 How to set it

Load the preset, confirm the values, reboot.

Companion app

1. Pick the **USA/Canada** preset — it loads the right band plan.
2. Confirm frequency **910.525**, bandwidth **62.5**, SF **7**, CR **5**.
3. Same values on **every** device — companion and repeater alike.

Repeater (CLI)

1. Set all four radio params in one command, then reboot to apply.

```
set radio 910.525,62.5,7,5
reboot
get radio # verify
```

! This is your address, not a tuning knob

Frequency, bandwidth and spreading factor together define the channel the whole mesh agrees on. Changing any of them on your node alone doesn't "improve" anything — it just removes you from the conversation. Match the network exactly, every time.

MESHCORE · FIELD CARD 02

Frequency

the freq in set radio 910.525,62.5,7,5 · = 910.525 MHz

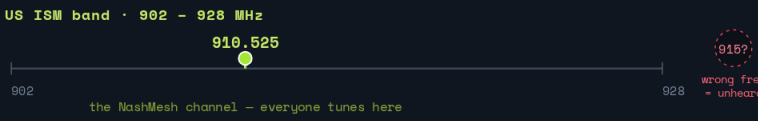
Which exact channel in the band the whole mesh tunes to. The most fundamental 'must match' of them all — wrong frequency and you're on a different station nobody else is listening to.

NASHMESH **910.525 MHz** | BAND **US 902-928** | RULE **must match exactly** | REBOOT **to apply**

1 What it is

The station everyone tunes to.

Frequency is the center of your radio channel — the precise point in the band where every transmission happens. Two radios on different frequencies are like two people on different stations: each is talking, neither hears the other. It's the **first** thing that has to match across the mesh.



Frequency is the exact point in the band where all transmissions happen. A node elsewhere is on a different station.

2 NashMesh value & how to set it

910.525 MHz, in the US 915 band.

Every node — companion or repeater — uses **910.525 MHz**, inside the US 902–928 MHz ISM band that the USA/Canada preset selects. Load the preset first; it puts you in the right band, then you confirm the exact frequency.

Companion app

→ The **USA/Canada** preset selects the band; confirm **910.525**.

Repeater (CLI)

→ Frequency is the first value in `set radio`; reboot to apply.

```
set radio 910.525,62.5,7,5
reboot
```

3 Why you don't touch it alone

It's the shared address.

! This is which channel — bandwidth and SF only shape it

Frequency is the shared address: get it wrong and nothing else matters, because no one else is listening there. It's tempting to nudge it to dodge interference, but moving it alone just removes you from the mesh. Different regions use entirely different bands (Europe sits at 868 MHz), which is why picking the right preset comes first.

MESHCORE · FIELD CARD 03

Bandwidth

the BW in set radio 910.525,62.5,7,5 · NashMesh = 62.5

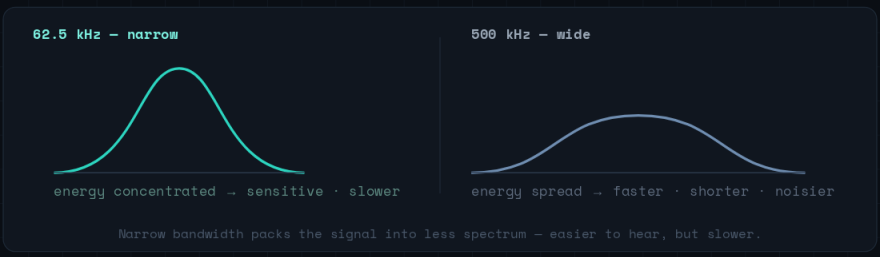
How wide a slice of spectrum your signal uses. Narrower means more reach and sensitivity but slower; wider is faster but shorter and noisier. The quiet partner to spreading factor.

NASHMESH 62.5 kHz | DIAL 62.5 → 500 kHz | NARROW = reach, slow | MATCH network-wide

1 What bandwidth is

The width of the radio channel your signal fills.

A **narrow** bandwidth concentrates the signal's energy into less spectrum, so the receiver hears it more easily — more range and sensitivity — but the data rate drops and packets take longer. A **wide** bandwidth spreads energy out: faster data, but shorter reach and it scoops up more background noise.



2 The dial

62.5 narrow & far → 500 wide & fast.

★ NASHMESH

62.5
KHZ · NARROW

reach	longest
speed	slowest
noise caught	least

Maximum sensitivity. Paired with fast SF7 for a reach/airtime balance.

125 / 250
KHZ · MID

reach	medium
speed	faster
noise caught	more

Common defaults on other meshes; quicker but less sensitive.

500
KHZ · WIDE

reach	shortest
speed	fastest
noise caught	most

High throughput, but pulls in more interference. Not used here.

3 Why NashMesh runs 62.5 kHz

Reach now, airtime managed by SF.

! Part of the channel — must match exactly

Like frequency and spreading factor, bandwidth defines the channel everyone shares. 62.5 kHz on every node, no exceptions — a different bandwidth simply can't be heard. Think of SF and bandwidth as a pair: NashMesh runs narrow bandwidth (for reach) with a fast spreading factor (for airtime), balancing the two.

MESHCORE · FIELD CARD 04

Spreading Factor

the SF in set radio 910.525,62.5,7,5 · NashMesh = 7

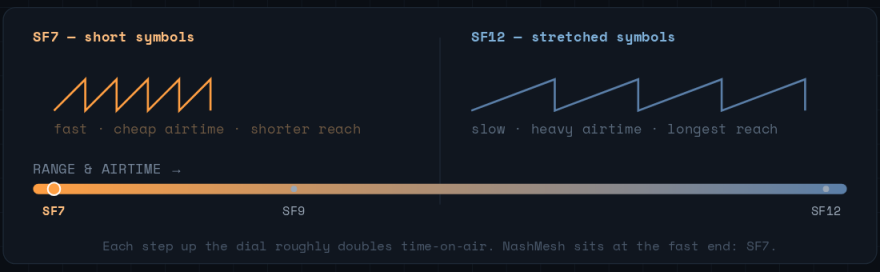
The range-vs-speed dial of LoRa. Higher reaches farther but clogs the air; lower is fast and cheap. Understanding it explains why NashMesh runs SF7.

NASHMESH SF7 | DIAL SF7 → SF12 | RULE each +1 ≈ 2× airtime | MATCH network-wide

1 What spreading factor is

How far each symbol is 'spread' across time.

A higher SF stretches every symbol over more time, so a receiver can pull a weaker signal out of the noise — that's the extra range and reliability. The price: the packet occupies the air far longer (**every step up roughly doubles airtime**) and the data rate drops. Lower SF is the reverse — quick, light on the channel, shorter reach.



Each step up the dial roughly doubles time-on-air. NashMesh sits at the fast end: SF7.

2 The dial

SF7 fast & cheap → SF12 far & slow.

★ NASHMESH	SF7	SF9	SF12
	FAST END	MIDDLE	MAX RANGE
reach	shorter	medium	longest
airtime	lowest	~4× SF7	~30× SF7
speed	fastest	medium	slowest
	Brief packets, channel stays open. Repeater density supplies the distance.	A compromise some sparse rural meshes use. Heavier on the air.	One slow packet can hog the channel. Wrong choice for a dense metro.

3 Why NashMesh runs SF7

In a dense mesh, cheap hops beat long ones.

With repeaters spread across the metro, no single node needs to shout across the whole city. You want **short, fast, low-airtime hops** that the repeater network relays onward — that keeps the shared channel free for everyone. The narrow 62.5 kHz bandwidth also claws back some of the sensitivity a low SF gives up, so SF7 stays reliable.

DO

- ✓ Run **SF7** — it's part of the NashMesh channel, same on every node.
- ✓ Trust the **repeaters** for distance, not one long heroic hop.

DON'T

- × Don't "bump SF for more range" on your own — a different SF can't be heard by the mesh.
- × Don't forget: higher SF = exponentially more airtime for **everyone**, not just you.

! SF is part of the channel, not a personal range boost

It's tempting to read "higher SF = more range" and turn it up. But SF is one of the values that must match across the mesh — raise it alone and you simply vanish from the network. Distance is the repeaters' job; your job is short, cheap packets.

MESHCORE · FIELD CARD 05

Coding Rate

the CR in set radio 910.525,62.5,7,5 · default = 5

The error-correction dial. More redundancy survives interference but slows every packet. NashMesh defaults to CR5 — and because CR is per-packet, you can run CR8 on a marginal node without breaking a thing.

DEFAULT **CR5 (4/5)** | DIAL **4/5 → 4/8** | EFFECT **airtime vs armor** | PER-PACKET · **mixable**

1 What coding rate is

How much error-correction padding rides with each packet.

LoRa adds redundant bits so a receiver can rebuild a packet that arrived partly garbled. Coding rate sets how much: **CR5 (4/5)** adds the least — one parity block per four data blocks — so packets are short and fast but tolerate the least corruption. **CR8 (4/8)** adds the most: tough against interference, but each packet takes far longer on the air.

2 The dial

CR5 light & fast → CR8 armored & slow.

★ DEFAULT

CR5

4 / 5

overhead	+25%
airtime	lowest
resilience	basic

NashMesh default now that most links are reliable. Short packets, free channel.

CR6–7

4/6 - 4/7

overhead	+50–75%
airtime	higher
resilience	more

Middle ground for moderately noisy links.

CR8

4 / 8

overhead	+100%
airtime	highest
resilience	max

For a marginal far node hardening its hop to the relay.

3 How NashMesh uses it

CR5 by default, CR8 where a link needs it.

Now that the mesh is dense, most links have a solid signal — so heavy error-correction on every packet is wasted airtime, and we moved the default down to **CR5**. The nodes still on **CR8** are the ones sitting far from the relay that reaches a major repeater: that extra armor keeps their one marginal hop alive, and because coding rate is per-packet they interoperate with everyone seamlessly.

DO

- ✓ Default to **CR5** on any node with a reliable link.
- ✓ Leave a **far / marginal** node on **CR8** to harden its hop — it stays fully readable by everyone.
- ✓ Drop back to CR5 once a link firms up, to give the airtime back.

```
set radio 910.525,62.5,7,5  # …,8
for a marginal node
```

DON'T

- ✗ Don't treat CR like frequency/SF — it does **not** need to match across the mesh.
- ✗ Don't leave a node on **CR8** once its signal is solid — that's airtime spent for nothing.

! Coding rate is per-packet — mixing is fine

Unlike frequency, bandwidth and spreading factor (which define the channel and must match), the coding rate rides inside each packet's own header. Any receiver reads it and adapts, so a CR8 node and a CR5 node hear each other perfectly. NashMesh moved the default from CR8 → CR5 as the mesh densified — but a node far from the relay reaching a major repeater can still run CR8 with zero downside to anyone else.

MESHCORE · FIELD CARD 06

Airtime

the shared resource every setting spends

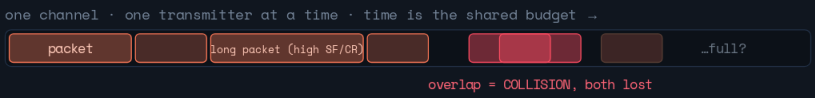
Not a setting — the thing all the settings are really about. Airtime is the finite slice of channel-time the whole mesh shares, and understanding it makes every other tuning choice make sense.

TYPE **shared resource** | RULE **one TX at a time** | DRIVER **SF · BW · CR · size** | FLOOD **cost × nodes**

1 What airtime is

Only one radio can talk on the channel at a time.

Every node on NashMesh shares **one** frequency, and radios can't talk over each other. So 'airtime' — how long each packet occupies the channel — is a finite, shared budget. When too many packets compete for the same moment, they **collide and are lost**, and the whole mesh slows down. Protecting airtime is protecting the network's capacity.



Every transmission claims a slice of the same timeline. Fill it up and packets collide.

2 What drives it

A few settings dominate how long a packet lasts.

WHAT MAKES A PACKET HOG MORE AIRTIME

Spreading factor (biggest lever)	each +1 ≈ 2×
Narrow bandwidth	halving BW ≈ 2×
Heavier coding rate (CR8 vs CR5)	≈ 1.6×
Bigger payload	linear
Re-flooding (every repeater repeats it)	× nodes that hear it

3 Why it's the thread through every card

Conserve airtime → faster, healthier mesh.

! Airtime is the budget every other setting spends

This is the thread tying all the cards together. SF, bandwidth, coding rate, advert interval, hop count, loop detection — every one of them is ultimately a decision about how much of the shared timeline you consume. A flood mesh repeats each packet across many nodes, so the cost multiplies. Keep packets short and adverts sane, and the whole network stays fast and responsive for everyone.

MESHCORE · FIELD CARD 07

Duty Cycle

`set dutycycle <percent>` · default 50%

A cap on how much of the time your radio is allowed to be transmitting. In the US there's no legal limit, so you leave it alone — but it's one of the most important settings in much of the world, and worth understanding.

FIRMWARE DEFAULT **50%** | RANGE **1–100%** | US **no legal limit** | EU **often 1–10%**

1 What it is

A ceiling on your transmit time.

Duty cycle is the share of time a radio may spend transmitting. After each transmission the firmware holds the radio silent for a while, so over the long run its on-air time stays under the cap. At **50%** a node spends at most half its time talking; at **10%** it must stay quiet about nine times as long as it just transmitted.

After each transmission the radio must stay silent long enough to keep its long-term airtime under the cap.

2 Why it exists

Shared spectrum, and in many places the law.

These are unlicensed bands — everyone shares them. To stop any one device from hogging the air, **many countries cap duty cycle by law**. In much of Europe the 868 MHz band is limited to 1% or 10% depending on the sub-band; exceed it and you're transmitting illegally and stepping on your neighbors. The firmware enforces the cap for you, so you stay legal without counting milliseconds.

US 902–928 MHz	**no fixed limit**
EU 868 MHz	**often 1% or 10%**
Elsewhere	**varies — check local rules**

3 What NashMesh does

Leave it at the default.

US 902–928 MHz has **no fixed duty-cycle limit** (the FCC governs the band with power and channel rules instead), so the firmware default of 50% is plenty of headroom and we don't change it. The point of this card isn't to tune anything — it's so you recognise the setting, understand the number, and know which knob keeps you legal if you ever operate where limits apply.

DO

- ✓ In the US, **leave it at the default 50%** — the firmware ships at 50% and the USA preset doesn't change it.
- ✓ Operating under EU-style rules? Set it to your **legal limit** (e.g. 10).
- ✓ Know the number exists and what it means — that's the whole goal here.

DON'T

- × Don't set **100** "for reliability" — duty cycle isn't a performance knob.
- × Don't assume your region has no limit without **checking** the local rules.

! A legal & courtesy setting — not a tuning dial

You'll see duty cycle sitting at 50% on a US node and wonder what it does. The short version: it caps how much of the time a radio is allowed to transmit, so no single device can monopolise a shared band. The US doesn't impose a fixed limit, so we leave it alone — but in much of the world it's the law, and that's why it's built in.

MESHCORE · FIELD CARD 08

Advert Intervals

`set advert.interval` · `set flood.advert.interval`

How often your repeater announces itself so the mesh can find it and build routes to it. NashMesh runs zero-hop 60 min and flood 3 h — tuned for fast discovery while the network grows.

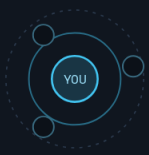
ZERO-HOP **60 min** | FLOOD **3 h** | FLOOD range **3–168 h** | COST **mesh-wide airtime**

1 Two kinds of advert

One is local; one floods the whole mesh.

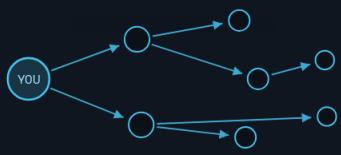
A **zero-hop** advert reaches only nodes in direct range — a quiet local hello. A **flood** advert is rebroadcast by every repeater that hears it, propagating across the entire mesh so far-off nodes learn a route to you. The flood is what makes you globally reachable, and that reach is exactly why it costs airtime.

ZERO-HOP — heard locally only



direct neighbours only · cheap

FLOOD — repeated mesh-wide



every repeater rebroadcasts it · cost multiplies

Zero-hop is a quiet local "I'm here." A flood advert propagates across the entire mesh — powerful, and expensive.

2 NashMesh values

60 minutes local, 3 hours flood.

Zero-Hop	`set advert.interval`	**60 min**
Flood	`set flood.advert.interval`	**3 h**

3 h is the firmware minimum — the loud end — chosen on purpose while the mesh is still growing, so new and roaming nodes discover routes fast. Matched to TennMesh.

3 The lever to relax as the mesh grows

Raise the flood interval later to reclaim airtime.

DO

- ✓ Keep **60 min / 3 h** while the mesh is still growing — discovery stays fast.
- ✓ As density climbs, **raise the flood interval** toward 12–24 h to give airtime back.

DON'T

- × Don't push flood **below 3 h** — it's the floor, and the airtime cost is mesh-wide.
- × Don't fuss over zero-hop — it's local and cheap; 60 min is fine.

! The flood interval is the airtime dial you'll relax as we mature

Because a flood advert is repeated by every repeater that hears it, its cost multiplies across the network — so this is the single biggest airtime lever an admin holds. Right now 3 h buys fast discovery for a growing mesh. Once routes are well-known and repeaters are permanent, stretching the flood interval reclaims that airtime with **no loss of reachability** — the mesh already knows where you are. Revisit it as the network fills in.

MESHCORE · FIELD CARD 09

Discovery & Adverts

name · position · key — signed

Why a fresh MeshCore mesh looks empty, and how nodes actually find each other — by trading signed adverts.

ADVERT **name · pos · key** | SIGNED **anti-spoof** | CLIENTS **on demand** | REPEATERS **~12 h flood**

1 Why the mesh looks empty

Nodes don't announce themselves until asked.

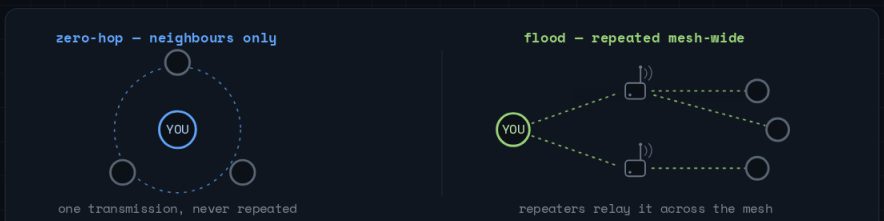
A new MeshCore mesh feels dead — by design. Nodes stay quiet; until two have each received the other's **advert**, they won't appear in each other's contacts, even on the same channel. An advert is a signed broadcast of your **name, position, and public key** — the handshake that makes private messaging possible.

On the same channel you can both use Public — but DMs need a traded advert first.

2 Zero-hop vs flood

Two ways to announce yourself.

Zero-hop broadcasts once to anyone in earshot and stops there. **Flood** is broadcast and then repeated by every repeater that hears it, so it reaches the whole mesh. Clients only advert when you tap the button; repeaters flood-advert on a timer.



Clients advert on demand; repeaters flood-advert about every 12 h (tune it on the Advert Intervals card).

3 Getting connected

Stir the mesh, then trade keys.

DO

- ✓ Fire a **zero-hop advert** to announce yourself to neighbours, or a **flood advert** to reach the wider mesh.
- ✓ Trade adverts **before** expecting to DM someone — that's the key exchange.
- ✓ If the mesh looks dead, **stir it**: send an advert or a Public message and watch contacts appear.

DON'T

- × Don't spam **flood adverts** — they cost airtime mesh-wide; keep repeaters near the default interval.
- × Don't expect to see nodes you've **never exchanged adverts with**, even on the same channel.
- × Don't read silence as a broken mesh — MeshCore is quiet on purpose.

! An advert is a signed introduction

Every advert carries your name, position, and public key, and it's signed so nobody can impersonate you. That public key is exactly what a DM encrypts to — which is why two nodes can't message privately until each has heard the other's advert. Discovery and encryption are the same handshake.

MESHCORE · FIELD CARD 10

Path Hash Mode

`set path.hash.mode 1` · NashMesh = 2-byte

How many bytes your repeater uses to stamp its ID into a packet's path. NashMesh recommends 2-byte to keep routing collision-free across a network this size; some operators run 3-byte to keep traffic more local.

RECOMMENDED **2-byte** | DEFAULT **0 (1-byte)** | NEEDS **firmware 1.14+** | PAIRS **loop detection**

1 The problem — 1-byte IDs collide

256 values can't uniquely name hundreds of nodes.

With a one-byte path ID there are only **256** possible values. On a mesh of hundreds of nodes, two repeaters inevitably share one — which muddies path diagnostics and triggers **false loop-detection hits**.

1-byte path ID → only 256 possible slots. With hundreds of nodes, two land on the same one.

same hash ID → collision

muddies path tracing · false loop-detection hits

2-byte = 65,536 slots → collisions effectively gone, routing stays clean.

2 The modes

More bytes = unique IDs, fewer hops.

Mode 0

1-BYTE

unique IDs	256
max hops	64
collisions	likely

Universal/legacy. Fine for tiny meshes; collides on a big one.

★ RECOMMENDED

Mode 1

2-BYTE

unique IDs	65,536
max hops	32
collisions	rare

Clean routing & reliable loop detection. Needs firmware 1.14+.

Mode 2

3-BYTE

unique IDs	16.7 M
max hops	21
collisions	none

Some run 3-byte on purpose — the 21-hop cap keeps floods more local. Trades reach for tighter scope.

3 Why NashMesh recommends 2-byte

Big enough to need it, current enough to run it.

DO

- ✓ Run at least **mode 1 (2-byte)** — the NashMesh recommendation, now that the mesh is on 1.14+.
- ✓ Companion: Experimental Settings → Default Path Hash Size → **2-Byte**.
- ✓ Want a tighter local footprint? **Mode 2 (3-byte)** is a valid pick — its 21-hop cap reins floods in.

```
set path.hash.mode 1
get path.hash.mode # 0=1B · 1=2B · 2=3B
```

DON'T

- ✗ Don't run multibyte on a node still on **firmware <1.14** — older builds drop those packets.
- ✗ Don't drop back to **1-byte** on a mesh this size — that's where the ID collisions start.

! What keeps routing — and loop detection — clean

With only 256 one-byte IDs, a big mesh inevitably has two repeaters sharing one, which muddies path tracing and triggers false loop-detection hits. Two-byte IDs (65,536 of them) make collisions vanish, so paths read true and loop detection can do its job. It's the NashMesh recommendation because the mesh is large enough to need it and current enough (1.14+) to run it — though some operators deliberately run 3-byte, trading reach for a lower hop ceiling that keeps floods more local.

MESHCORE · FIELD CARD 11

Loop Detection

`set loop.detect <level>`

A one-page brief for repeater admins: what packet loops are, what the `loop.detect` setting actually does, and the level we recommend running mesh-wide.

FIRMWARE ≥ 1.14 | DEFAULT **off** | SCOPE **flood packets** | DECISION **per-repeater, local**

1 The problem — one bad node, whole-mesh storm

A repeater on broken/forked firmware can mangle a flood packet and re-emit it as "new."

Every repeater that hears a flood packet rebroadcasts it. If a faulty node keeps changing a packet so it looks fresh, the network re-floods it again and again — **up to the 64-hop ceiling** — saturating airtime, draining battery on every neighbor, and drowning out real traffic.

A single misbehaving repeater turns one packet into a broadcast storm.

2 What loop.detect does

It reads the packet's `path` — the list of repeater IDs that already touched it.

Each repeater appends its own ID/hash to a flood packet before forwarding. With `loop.detect` on, a repeater first counts **how many times its own ID already appears in that path.** Too many = the packet has looped back through it → **drop it, don't amplify.**

src → A4 → C7 → 7F → B9 → C7 → ?

"C7" sees its own ID twice in the path → LOOP → packet dropped

Trigger threshold depends on the chosen level *and* the path-hash size (1 / 2 / 3-byte). See below.

3 The four levels

Number of times your own ID must appear in the path before the packet is dropped.

	OFF DEFAULT	MINIMAL VERY LAX	★ RECOMMEND MODERATE BALANCED	STRICT AGGRESSIVE
1-byte	–	4+	2+	1+
2-byte	–	2+	1+	1+
3-byte	–	1+	1+	1+
	No checking. Relies only on the dedup cache.	Rarely fires on a hop-limited mesh. Barely above "off."	Catches real loops; false-positive risk stays low.	Drops on a single hit – risky on 1-byte paths (see below).

Why not just run strict? — the 1-byte collision problem

A 1-byte path hash has only **256 possible values**. Across a mesh of ~500 nodes, that's roughly **two different nodes sharing every prefix**. So on a 1-byte path, "my ID appears once" might just be a **prefix twin**, not a loop. Under **strict**, that one collision **false-drops legitimate traffic.**

4 Recommended for our mesh

A sensible default — not a mandate.

DO

- ✓ Run **`moderate`** on repeaters that are on firmware **≥ 1.14**.
- ✓ Prioritize **backbone / high-traffic** nodes — they're the amplifiers.
- ✓ Keep **co-located / sibling** repeaters on a site matched to the same level.
- ✓ Watch relay counts for a day after enabling; if traffic drops, fall back to **`minimal`**.

DON'T

- × Push everyone to **`strict`** while the mesh is still 1-byte — mass false drops.
- × Treat it as a **cure**. It contains storms; it doesn't fix the node causing them.
- × Enable on pre-1.14 firmware (the setting won't exist).

```
set loop.detect moderate
get loop.detect  # verify
```

> **! It's a backstop, not a fix**
>
> loop.detect stops a healthy repeater from *amplifying* a storm. The real cures are upstream: keep firmware on current official builds, avoid mystery forks, and correct loud / TX-RX-imbalanced nodes. Loop detection just keeps the rest of us from making it worse.

MESHCORE · FIELD CARD 12

Multi-Acks

`set multi.acks 1` · **NashMesh = 1**

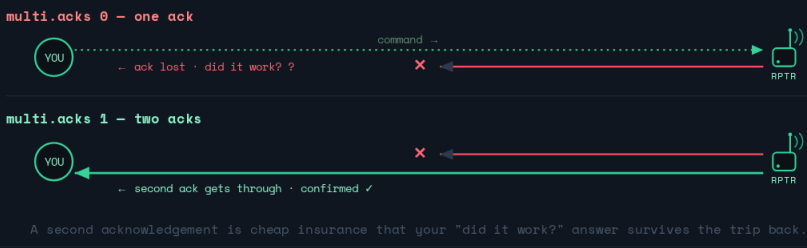
Also called “double ACKs.” Makes a repeater send two delivery confirmations instead of one — a small reliability boost that mostly pays off when you’re administering a node remotely.

`NASHMESH 1 (on)` `DEFAULT 0 (off)` `HELPS remote admin` `COST tiny`

1 The problem — a lost ACK leaves you guessing

You sent a command. Did it land?

When you send a command to a repeater — say, changing a setting from home — the node acts on it and sends back an **ACK** to confirm. Over several hops, that single confirmation can die on the return path. The repeater did the work, but you never hear back, so you're left unsure whether to resend.



A second acknowledgement is cheap insurance that your "did it work?" answer survives the trip back.

2 What multi-acks does

Sends two confirmations instead of one.

DO

- ✓ Turn it **on (1)** on every repeater — NashMesh standard.
- ✓ Set it **before** the node goes up a tree/pole where you can't reach it.

```
set multi.acks 1
get multi.acks # 0 = off · 1 = on
```

DON'T

- ✗ Don't expect it to help **user chat** — it's about confirming **your** remote commands landed.
- ✗ Don't leave it **off (default)** on a hard-to-reach node and then wonder if your config stuck.

! This one's for you, not the network

Multi-acks ("double ACKs") mostly improves *remote administration* — managing a repeater you can't physically touch. The node sends two confirmations instead of one, so when you push a setting from miles away, the answer is far more likely to make it back across all those hops. Cheap, and worth it on every NashMesh repeater.

MESHCORE · FIELD CARD 13

RX Delay

`set rxdelay 3` · NashMesh = 3

A clever signal-priority trick: it briefly holds weak-signal copies of a flood packet so strong-signal nodes forward first — letting the mesh prefer clean paths and drop redundant retransmits on its own.

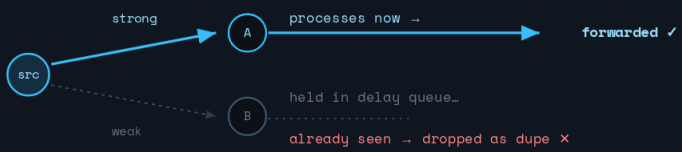
NASHMESH 3 | DEFAULT 0 | STATUS experimental | EFFECT prefers clean paths

1 The problem — weak repeats add noise, not value

Every node that hears a flood wants to repeat it.

In a flood, **every** repeater that hears a packet rebroadcasts it — including nodes that barely caught it. Those weak-signal repeats spend airtime and add congestion without improving delivery, since a stronger node has usually already carried the packet onward.

one flood packet, heard by two repeaters at different signal strength



Strong-signal copies forward first; weak copies wait, then get suppressed as duplicates.

2 What rxdelay does

Delays weak copies so strong ones win.

It adds a short processing delay scaled to **signal quality**. Strong-signal copies are handled immediately; weak ones wait in a queue. By the time a weak copy would be processed, the packet has usually already propagated — so it's recognized as a duplicate and dropped before it ever wastes airtime being retransmitted.

DO

- ✓ Set **rxdelay 3** on every repeater — NashMesh standard.
- ✓ Think of it as **automatic path preference** — no manual routing needed.

```
set rxdelay 3
get rxdelay # experimental · range 0–20
```

DON'T

- × Don't confuse it with **`txdelay`** — that spaces out *transmits*; this prioritizes *which copy* forwards.
- × Don't crank it sky-high — too much delay slows legitimate forwarding.

! The mesh sorts its own best paths

rxdelay and txdelay work as a pair: txdelay keeps simultaneous repeats from colliding, while rxdelay lets the **cleanest-signal** node forward first and quietly suppresses the weak, redundant copies. The result is that well-placed nodes naturally carry the traffic — self-organizing routing with zero manual config.

MESHCORE · FIELD CARD 14

TX Delay

set txdelay / direct.txdelay · by neighbor count

Collision avoidance for retransmits. When several repeaters hear the same packet, this staggers when they repeat it so they don't fire at once. NashMesh scales it to how many neighbors a node sees.

TUNED BY **neighbor count** | DEFAULT **0.5 / 0.2** | RANGE **0–2** | SCOPE **flood + direct**

1 The problem — simultaneous repeats collide

Two radios transmitting at once cancel each other.

When multiple repeaters hear the same flood packet and rebroadcast it at the same instant, their signals **collide and the packet is lost**. The denser the cluster of repeaters that all hear each other, the more often this happens.

Each repeater waits a random slice before repeating. The more neighbors compete, the wider the window needs to be.

2 NashMesh's neighbor-based table

More neighbors → wider random window.

NEIGHBORS YOUR NODE SEES	TXDELAY	DIRECT.TXDELAY
0 – 1	**0.3**	0.1
2 – 4	**0.5**	0.3
5 – 9	**1**	0.5
10 – 14	**1.5**	1
15 +	**2**	2

Check your count with the neighbors command, and revisit these as the node sees more over time. direct.txdelay is lower because addressed traffic has far fewer nodes competing to forward it.

3 Applying it

Set both values for your tier, revisit as you grow.

DO

- ✓ Match the table to your node's **current neighbor count**.
- ✓ **Re-tune** as the node sees more neighbors — a growing cluster needs a wider window.

```
set txdelay 0.5
set direct.txdelay 0.3
# the 2-4 neighbor tier
```

DON'T

- × Don't set a **lonely** node to a wide window — it just adds needless latency.
- × Don't leave a **dense-cluster** node at a tiny window — that's where collisions pile up.

> **! Scale the spacing to the crowd**
>
> A node that hears one neighbor almost never collides, so it barely needs to wait. A node surrounded by fifteen all hear the same packet and would stomp on each other — so it needs the widest window. That's why NashMesh tunes by neighbor count instead of one fixed value, and why it's worth revisiting as the mesh fills in around you.

AGC Reset Interval

`set agc.reset.interval 4` · **NashMesh = 4**

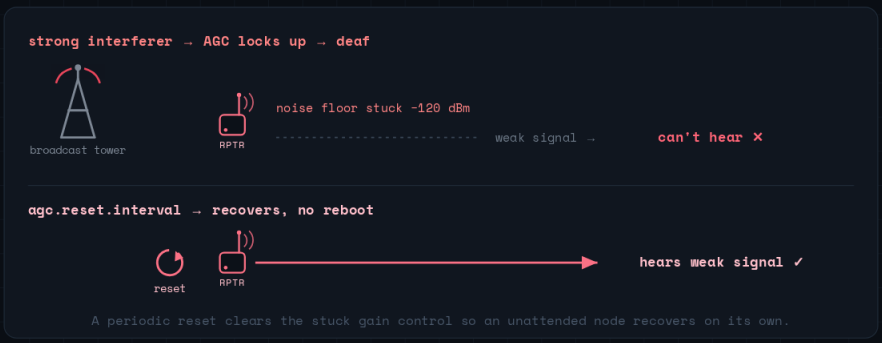
Keeps a repeater from going quietly "deaf." The radio's automatic gain control can lock up after a strong nearby signal; this periodically resets it so an unattended node recovers without a reboot.

NASHMESH **4 sec** | DEFAULT **0 (off)** | STEP **multiples of 4** | FOR **elevated nodes**

1 The problem — a deaf repeater that only a reboot fixes

Strong RF can lock up the radio's gain control.

The SX1262's automatic gain control can get **stuck** when hit by a strong out-of-band signal — a broadcast tower, a cell site, a nearby handheld. When it does, the noise floor pins at –120 dBm and the repeater goes **deaf to weak signals** until it's power-cycled. On an unattended hilltop node, that's a dead repeater nobody notices.



2 What it does

Periodically resets the gain control automatically.

Setting an interval makes the firmware reset the AGC on a timer, so it recovers on its own — no reboot, no climb. The gain control already resets after every transmission, so this is really insurance for nodes that go long stretches just **listening**. NashMesh uses **4 seconds**: the most aggressive recovery, and a very low-cost operation.

DO

- ✓ Set **agc.reset.interval 4** — NashMesh standard, the most aggressive recovery.
- ✓ Prioritize it on **elevated / hilltop / tree** nodes near broadcast or cell sites.

```
set agc.reset.interval 4
# seconds · 0 = off · rounds to multiples of 4
```

DON'T

- × Don't leave it at **0 (off)** on an unattended node — deafness needs a reboot to clear.
- × Don't worry about cost — it's cheap; some networks use larger values (~500) more conservatively.

! Matters most on hard-to-reach nodes

The radio's gain control already resets after every transmission — so this only matters for nodes that sit quietly listening for long stretches. That's any unattended repeater you can't easily power-cycle — rooftop, mast, hilltop or tree-mounted, especially near other RF sources — exactly where a stuck AGC would otherwise silently kill reception until someone makes the trip to reboot it. Cheap insurance: set it and forget it.

MESHCORE · FIELD CARD 16

Reading Your Signal

RSSI · SNR · Noise Floor

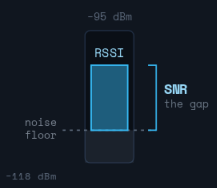
The three numbers your node reports for every packet it hears — what each one means, what counts as healthy, and which one actually tells you if a link is good.

SOURCE **stats-radio / app** | UNITS **dBm · dB** | KEY METRIC **SNR** | LoRa **decodes below noise**

1 The three numbers

Signal, the noise it sits in, and the gap between them.

RSSI is how strong the incoming signal is. The **noise floor** is the ambient RF racket your receiver sits in. **SNR** is how far the signal stands above that noise — roughly RSSI minus the noise floor — and it's the single best gauge of link quality. All three are in dBm except SNR, which is a ratio in dB.



- **RSSI** — how strong the signal arrives (dBm; closer to 0 = strong)
- **Noise floor** — the ambient RF din at your site (lower = quieter)
- **SNR** — how far signal clears noise (dB) · the one to watch

SNR ≈ RSSI minus the noise floor — the height of the signal above the din.

2 What's good, what's not

Bands for a MeshCore / SF7 link.

METRIC	GOOD	OK / EDGE	PROBLEM
RSSI (dBm)	≥ -100	-100 to -118	≤ -120
SNR (dB)	≥ 0	0 to -7 *	< -7.5 *
Noise floor (dBm)	≤ -120	-120 to -110	≥ -105

* LoRa decodes **below** the noise floor — negative SNR still works. A higher spreading factor lowers that limit (SF12 ≈ -20 dB); at NashMesh's SF7 the floor is about -7.5 dB.

3 Using the numbers

How to read a reading — and fix a weak one.

READING IT

- → Weak RSSI but **decent SNR** = far but clean. Normal for a long rural hop — it'll work.
- → Strong RSSI but **low SNR** = close but noisy. Something local is interfering.
- → Noise floor **pinned at −120** and never moving? Suspect a deaf radio — see the AGC Reset card.

FIXING A BAD LINK

- ↑ Raise the **signal**: height, better antenna, shorter path, clear line of sight.
- ↓ Lower the **noise**: move away from interferers, get the antenna clear of local RF.

```
stats-radio  # noise floor · last RSSI/SNR (serial)
neighbors    # who you hear, with SNR
```

! Watch SNR, not RSSI

RSSI alone can fool you — a strong signal in a noisy spot still decodes badly. SNR already folds the signal and the noise together, so it's the truest read on whether a link is healthy. And remember LoRa's trick: a negative SNR isn't automatically broken — the radio can still pull signal out from under the noise, especially at higher spreading factors.

Access & Remote Admin

who can read · who can change

Two passwords decide what anyone can do to a node — and the companion app lets you do it from across the mesh, no climb required.

`ADMIN full control` `GUEST read-only` `DEFAULT password` `REACH BLE / mesh`

1 Who can read, who can change

Two passwords decide everything.

Every repeater and room server ships with the admin door **unlocked**: the default admin password is the literal word **password**, and it's publicly known. Until you change it, anyone can reconfigure your node. A separate **guest** password controls read-only access.

ADMIN — full control

- ✓ Default is the literal word **password** — publicly known. **Change it first.**
- ✓ Unlocks **everything**: radio, name, location, passwords, reboot.

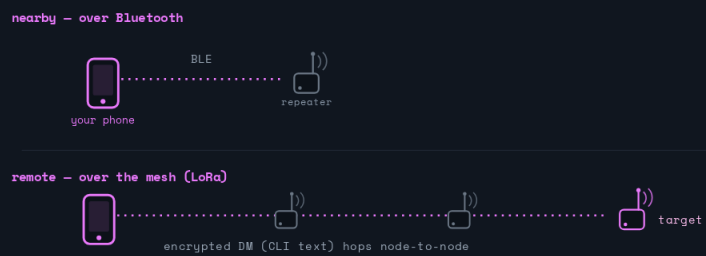
GUEST — read only

- ✓ Leave it **blank** on repeaters — anyone may read **stats, telemetry, neighbours**.
- ✓ Changes **nothing**. (Room servers default **hello** so people can post.)

2 Managing it remotely — no climb

From the app, near or far.

The companion app's **Remote Management** reaches a node two ways: over **Bluetooth** when you're beside it, or across the **mesh** over LoRa when you're not — routed through your companion node, no internet involved.



No internet, no climb — the app reaches the node over Bluetooth up close or across the mesh from afar.

3 Doing it safely

Lock it down, then manage from the couch.

DO

- ✓ Set a real **admin password** before the node ever leaves your bench.
- ✓ Manage **over the mesh** to skip the climb; turn on **multi-acks** for reliable confirms (see the Multi-Acks card).
- ✓ Leave **guest blank** — transparent stats for everyone, zero risk.

DON'T

- × Don't keep the default **password** — anyone could reconfigure your node.
- × Don't lose it — the only reset is a **re-flash**, i.e. a trip up the tree.
- × Don't fire a config you can't **confirm landed** over a long, weak path.

! Remote admin is a DM under the hood

Managing a node isn't a separate channel — it's CLI text wrapped in an ordinary **encrypted DM** to that node, flagged as a command. So it's private by the node's own key and self-routing for free: it floods to find the node, then follows the learned path back. The admin password is the only thing standing between “read” and “change.”

Owner Info

`set owner.info <text>`

A little contact card baked into your repeater so anyone on the mesh can reach you when something goes wrong with it. Unattended nodes especially need it.

COMMAND **set owner.info** | FIRMWARE **1.12+** | VISIBLE **when guest blank** | TIP **call sign + contact**

1 Why set it

Your node can't call you when it breaks.

Your repeater is unattended and probably somewhere annoying to reach. When it misbehaves, other operators can see *which* node is the problem but have no way to reach *you* — unless your contact is on the node itself. Owner info is that note, readable by anyone who can log in as a guest.

A tiny contact card baked into the node — so people can reach you, not just your silent repeater.

2 What to put, and how

Short, useful, and formatted with | breaks.

Keep it brief: a call sign or name, a contact method, and maybe a site name or rough location. The **|** character becomes a line break, so you can lay it out cleanly. It's visible to guests — which means everyone, when you leave the guest password blank (the usual community setup).

DO

- ✓ Include a **call sign or name** plus one reliable contact (email / handle).
- ✓ Use **|** to break it into clean lines; add a rough site name if useful.
- ✓ Keep **guest blank** so it's actually visible to the people who'd need it.

```
set owner.info N0CALL |
n0call@example.com | Mt Juliet
```

DON'T

- × Don't put **sensitive** details (home address, phone) — it's public to any guest.
- × Don't leave it **blank** on a node nobody can physically get to.
- × Don't forget it needs **firmware 1.12+** to be available.

! The difference between a heads-up and a mystery

When your repeater drifts off frequency, starts flooding, or goes deaf, the operators who notice have no way to tell you — unless your contact is right there on the node. Owner info turns "some node is misbehaving" into "hey, your repeater's acting up." It costs one command and saves everyone, including you, a lot of guesswork.

Direct Messages

private 1-to-1 · end-to-end encrypted

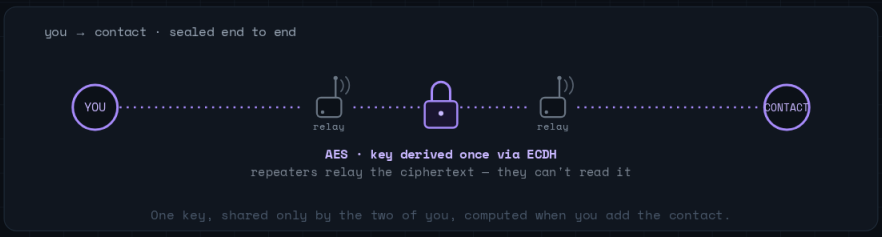
How a MeshCore DM differs from a channel: it's encrypted for one contact only, and it reaches them by flooding once, then following the route it learns.

ENCRYPTION **AES + ECDH** | KEY **per contact** | ROUTING **flood → path** | VS CHANNELS **1-to-1**

1 Sealed to one key

A DM is encrypted for one contact — not the channel.

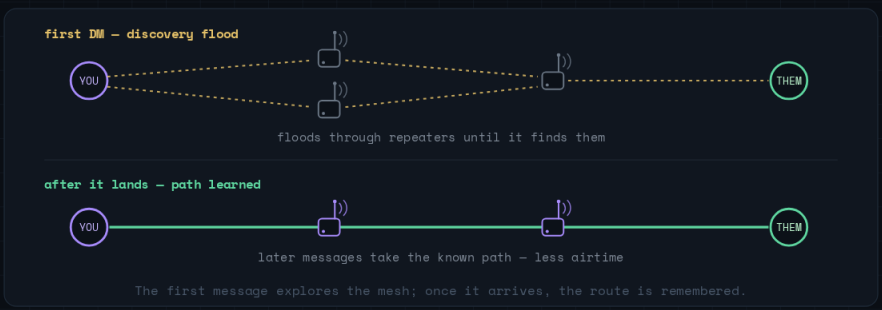
Open a contact and your message is encrypted with a key shared only by the two of you. MeshCore derives that key **once** — when you add the contact from their advert — using **ECDH**, then encrypts every message to it with **AES**. A built-in check (MAC) means a tampered packet is rejected. Repeaters carry the ciphertext across the mesh but can't read a word of it.



2 Finding them — flood, then path

The first message explores; the rest take the known route.

The first time you DM someone your node doesn't know the route, so the message **floods** outward through repeaters until it reaches them (within the 64-hop ceiling). Once it lands and a reply comes back, the **path is learned** — the chain of repeaters — and later messages follow that direct path instead of flooding. That's why the first message to a new contact can be slow, then the thread speeds up. Repeaters do the forwarding; client nodes don't relay.



3 DMs vs channels

Per-contact keys vs one shared key.

A **channel** uses one shared key: handy for a group, but everyone holding the key reads everything. A **DM** has its own per-contact key, so it's the only truly private, one-to-one path on the mesh.

DO

- ✓ Add a contact from their **advert** first (it carries their key), then tap to chat.
- ✓ Use DMs for anything private — including **remote admin**, which rides the same encrypted DM.
- ✓ Expect the **first** message to a new or quiet contact to lag while the path is found.

DON'T

- ✗ Don't assume a **channel** is private — everyone with the key reads everything on it.
- ✗ Don't pass a contact around carelessly; privacy is only as good as your contact list.
- ✗ Don't expect delivery with **no path** and no repeater bridging you — a DM still needs a route.

> **! Remote admin is just a DM**
>
> Administering a node over the air uses the very same encrypted direct message — the CLI text rides inside an ordinary DM packet, flagged as a command rather than chat. So the per-contact key that keeps your messages private, and the flood-then-path routing that delivers them, are exactly what protect and carry your **`set`** commands too.

MESHCORE · FIELD CARD 20

Channels

public · #hash · private

Group chat on the mesh. Every channel is one shared key that everyone on it can read — the only real question is who holds that key, and how they got it.

TYPE **group · shared key** | ROUTING **flood · no ACK** | NASHMESH **#tn-* · #bna-*** | PRIVATE **own PSK**

1 How a channel works — one shared key

Everyone holding the key reads everything.

A channel is a group room secured by one symmetric key (AES) that every member shares. Anyone with the key reads and sends; anyone without it just hears noise. Unlike a DM, a channel message is a **flood** — it spreads across every repeater instead of following a path, and there's **no delivery confirmation**. You can see which repeaters relayed it, but not whether anyone received it.

2 Three kinds — Public, #hash, private

Same mechanism, very different privacy.

All three ride the same group encryption. What separates them is entirely the **key**:

PUBLIC	#HASH	PRIVATE
SHIPS BY DEFAULT	NAME = KEY	YOUR SECRET KEY
key — **published**	key — **from the name**	key — **random**
read — **anyone**	read — **anyone who knows it**	read — **key holders only**
for — discovery, chatter	for — open topic / regional	for — sensitive groups

Same AES underneath — what differs is who holds the key and how they got it.

3 NashMesh channels

Join by name — the name sets the key.

Type these in your app and MeshCore derives the shared key for you. They're all open **#** channels, so anyone who knows the name is in.

REGIONAL	
#tn-middle	all of Middle Tennessee
#tn-davidson	your county — #tn-<county> for any nearby one
TOPIC	
#tenntalk	statewide general chatter
#ham	amateur (ham) radio operators
BOTS & TESTING	
#bna-wx	weather chatter + the NashMesh wx bot
#bna-bot · #bot	automated traffic from bots
#test	human-to-human "can you hear me?" checks

> **! These are open by design**
>
> Every NashMesh channel above is a **#** channel — the name is the key, so anyone who knows it can join and read along. That's exactly what you want for community traffic. Keep anything sensitive on a **private** channel (your own key, shared out-of-band) or send a **DM**.

MESHCORE · FIELD CARD 21

Weather Bot

bna-wx-bot · lives on #bna-wx

Message any of these on #bna-wx and the bot answers. **wx** is the US (NOAA); **gwx** is worldwide (Open-Meteo). Leave off a place and it defaults to Nashville.

HOME **Nashville** | US **wx** | WORLD **gwx** | ON **#bna-wx**

1 wx — US weather

aliases weather, wxa · source NOAA

Bot's home (Nashville)	`wx`
City	`wx nashville`
City, state	`wx franklin, tn`
ZIP code	`wx 37130`
Coordinates	`wx 36.16,-86.78`
Bare state → capital	`wx tennessee`
Tomorrow	`wx 37130 tomorrow`
N-day (3 · 5 · 7)	`wx 37130 3d` · `wx franklin 5d` · `wx nashville 7d`
Hourly	`wx nashville hourly`
Full active alerts	`wx 37130 alerts`
Foreign place	`wx paris` → use `gwx`

2 gwx — worldwide

aliases globalweather, gwxa · source Open-Meteo

City	`gwx tokyo` · `gwx london`
City, country	`gwx paris, france`
Bare country → capital	`gwx france` · `gwx japan`
Coordinates	`gwx 51.5,-0.13`
Tomorrow / N-day / hourly	`gwx tokyo tomorrow` · `gwx berlin 5d` · `gwx tokyo hourly`

3 rain · snow · nowcast

precip nowcast — next ~2 h, with amount

Rain at home	`rain`
Rain at a place	`rain 37130` · `rain murfreesboro` · `rain 36.16,-86.78`
Snow (shows depth)	`snow` · `snow denver`
Neutral (rain or snow)	`nowcast` · `nowcast 37130`

4 sun · sky · space weather

ham & solar extras

sun — sunrise / sunset at the bot's location

moon — moon phase + rise / set

aurora (kp) — aurora / KP-index forecast
`aurora` · `aurora 37130` · `kp seattle`

aqi (air, airquality) — air quality index
`aqi nashville` · `aqi vancouver canada` · `aqi 36.16,-86.78`

hfcond (hf) — HF band propagation, ham radio
`hfcond` · `hf`

solar — solar conditions + HF band status

solarforecast (sf) — solar-panel production forecast
`sf seattle` · `sf 37130 10 0 37 (watts, azimuth, tilt)`

satpass — satellite pass predictions
`satpass iss` · `satpass noaa15` · `satpass 25544` · `satpass 27607 visual`

! No location? You get Nashville

Every command falls back to the bot's home. Point it anywhere with a city, ZIP, or **lat,lon**. A bare state or country returns its capital; a foreign place on **wx** nudges you over to **gwx**.

MESHCORE · FIELD CARD 22

Test Bot

bnabot · lives on #bot · #bna-bot

Reachability and routing checks. Message a command on #bot or #bna-bot and BNABot answers with connection info, decoded paths, and network lookups.

CHECK **test · ping** | PATHS **tracer · path** | ON **#bot · #bna-bot** | IDS **2-char hex**

1 Core checks

Is the bot hearing me?

test (t) — confirms the bot got your message; reports direct/routed, SNR, and path — the bread-and-butter reachability check
`test · test hello world (echoes a phrase back)`

ping — simplest liveness check
`ping → Pong!`

help — lists all commands, or details for one
`help · help prefix`

cmd — compact list of every available command
`cmd`

2 Path & link diagnostics

The meaty part for network work.

path (decode, route) — decodes and shows the full routing path your message took — every intermediate node
`path`

trace — link trace along a given path; the return may not be heard by the bot, so it's one-directional
`trace 01,7a,55 (comma-sep 2-char hex; none = your incoming path)`

tracer — like trace, but auto-builds a round-trip (01,7a,55 → 01,7a,55,7a,01) so the bot hears the reply — the one you want
`tracer 01,7a,55 · tracer (uses your incoming path)`

multitest (mt) — listens 6 s and collects every unique path it hears — how many routes are live
`multitest`

> **! Trace timing**
> Each attempt waits **`base 1.0 s + hops × 0.5 s`**, then retries twice. Tune under **`[Trace_Command]`**: maximum_hops, trace_mode (one-/two-byte), timeout_base / per_hop, retry count & delay.

3 Network utility

Look up the rest of the mesh.

prefix <XX> — look up repeaters by their two-char prefix
`prefix 1A · prefix 2B all · prefix refresh · prefix free`

stats — bot usage over the last 24 h
`stats · stats messages · stats channels · stats paths`

channels — list/inspect hashtag channels on the network
`channels · channels list · channels #general`

> **! Read the reply, not just "it replied"**
> A **`test`** answer tells you **how** it came back: **direct** (no repeaters — a strong local link) or **routed** with the exact hops, plus the SNR — so it doubles as a quick coverage check. For real path work reach for **`tracer`** over **`trace`**: it forces a round trip so the bot's own radio actually hears the answer come back.

MESHCORE · FIELD CARD 23

Troubleshooting

can't hear · can't be heard

A quick decision path when a node won't talk to the mesh — from the most common cause to the rarest. Work down; stop at the first miss.

START **radio match** → THEN **discovery** → THEN **signal** → LAST **deafness**

1 Work down the list

Stop at the first thing that's wrong.

1. **Same channel?** freq · BW · SF · preset identical to the mesh
   ↓ no — you're invisible. Fix it on **Radio Settings**.
2. **Traded adverts?** you only see nodes you've swapped keys with
   ↓ no contacts — send a zero-hop advert. See **Discovery & Adverts**.
3. **Signal above the floor?** RSSI / SNR healthy, not buried in noise
   ↓ weak — height, antenna, clear the path. See **Reading Your Signal**.
4. **Radio gone deaf?** noise floor pinned at -120 dBm, hearing nothing
   ↓ yes — AGC reset or reboot. See **AGC Reset**.
5. **One-way link?** they hear you, but you don't hear them
   check your **RX**: antenna, coax, local interference.

2 Symptom → first move

The fastest read on where to look.

SYMPTOM	FIRST MOVE
No one hears me	confirm the preset / radio params match exactly
Contacts list is empty	send an advert — the mesh is quiet by design
Heard, but barely	raise the antenna before anything else — height beats power
Was fine, now deaf	reboot or AGC reset; suspect a pinned noise floor
They hear me, I don't	the problem is on your receive side

3 Still quiet after all that?

When the checklist passes but it's silent.

STILL QUIET?

- ✓ Run a **traceroute** / check the path — see which repeaters are actually carrying you.
- ✓ Pull up the coverage map at **nashme.sh** to spot dead or one-way zones near you.
- ✓ Post on **#test** and ask a known-good node for a signal report.

QUICK WINS

- ✓ **Reboot** first — it clears a deaf AGC and stale paths in one move.
- ✓ **Raise the antenna** a few feet before adding power; height beats watts.
- ✓ Re-send an **advert** after any move so neighbours re-learn your path.

> **! Work top-down — it's almost never the hardware**
>
> The overwhelming majority of "dead node" problems are a **channel mismatch** or a missing advert, not a broken radio. Start at the top of the list and stop at the first thing that's wrong; you'll fix most issues before you ever think about antennas or firmware.

Join us on Discord

Coordinate nodes, ask for a signal report, and meet the operators behind the mesh.

discord.gg/busguku7St

point your camera at the code to join

THE MESH ONLINE

nashme.sh